МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Информационный поиск»

Поисковый робот

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 08.10.2026
Оценка:

Москва, 2026

Содержание

1 Постановка задачи 2

2 Метод решения 2

2.1 Источники и входные данные 2

2.2 Очередь и сохранение документов 2

2.3 Проверка изменений 3

3 Результаты 3

3.1 Запуск и автотесты 3

3.2 Пробная обкачка 3

3.3 Репозиторий 5

4 Вывод 5

Приложение А. Примеры запросов к MongoDB 6

1 Постановка задачи

Для корпуса игровых публикаций из первой лабораторной работы требуется разработать поисковый робот:

- реализовать обкачку документов;
- принимать при обычном запуске один аргумент — путь к YAML-файлу с данными базы в секции `db` и параметрами робота в секции `logic`, в том числе задержкой между запросами;
- сохранять в базе нормализованный URL, сырой HTML, название источника и дату загрузки в формате Unix timestamp;
- после остановки продолжать обход с незавершённого документа;
- периодически проверять загруженные страницы и обновлять сохранённые документы только при изменении содержимого.

2 Метод решения

2.1 Источники и входные данные

В качестве корпуса выбраны публикации восьми игровых сайтов. Русскоязычные источники — PlayGround, StopGame, IXBT.games, Игромания и GameMAG; англоязычные — PC Gamer, Eurogamer и GamingOnLinux. Входом служит UTF-8 TSV-файл: короткий `urls.txt` из первой лабораторной либо объединённый инвентарь sitemap. Последний содержит 1 055 674 URL-кандидата, но это ещё не число пригодных документов.

Файл читается построчно и импортируется в MongoDB партиями, без загрузки всего списка в память. URL очищается от параметров отслеживания, его хост сверяется с разрешённым списком. Пробная выборка воспроизводима по заданному seed; порядок очереди перемешивается устойчивым хешем URL.

2.2 Очередь и сохранение документов

Для хранения выбрана MongoDB. Очередь и содержимое страницы разделены по коллекциям, чтобы просмотр результатов разбора не загружал большой HTML:

Коллекция	Содержимое
`frontier`	URL, источник, состояние, время следующей попытки, число попыток, HTTP-заголовки и последняя ошибка
`raw_pages`	Нормализованный URL, источник, Unix-время загрузки, сырой HTML и SHA-256 ответа
`documents`	Результат разбора, HTTP-метаданные, время загрузки и ссылка `raw_page_id` на исходный HTML
`crawler_meta`	Метка импорта инвентаря и паузы для отдельных сайтов

Ключ `_id` у заданий и документов равен нормализованному URL, поэтому повторная загрузка не создаёт дубликатов. Сначала сохраняется сырой HTML, затем результат разбора и только после этого завершается задание очереди. Если выполнение прервётся между этими шагами, при следующей попытке согласованность записей проверяется по SHA-256.

Робот атомарно выбирает доступный URL и отмечает его как обрабатываемый. Затем он проверяет `robots.txt` и делает запрос с паузой для каждого хоста. Редиректы проверяются по отдельности: переход за пределы источника не выполняется. При перезапуске незавершённые URL возвращаются в начало очереди. Механизм рассчитан на один процесс робота.

2.3 Проверка изменений

При повторном запросе робот использует сохранённые `ETag` и `Last-Modified`. Ответ `304` оставляет HTML неизменным. Если сервер возвращает `200`, вычисляется SHA-256 исходного ответа: при совпадении хеша обе записи не перезаписываются. Для изменённого ответа обновляются `raw_pages`, `documents` и `fetched_at`. Если одна из двух записей отсутствует либо их хеши расходятся, робот повторно получает HTML и восстанавливает согласованное состояние. Время каждой проверки хранится в `frontier.last_checked_at`. При ошибках попытка откладывается: `404`/`410` и запрет `robots.txt` проверяются реже, для `429` учитывается `Retry-After`. Пауза одного источника хранится в MongoDB и не останавливает другие сайты. У завершённого задания время следующей проверки задаётся полем `available_at` через `revisit_seconds` после последней проверки. Обычный запуск берёт новые URL и повторы их временных ошибок. Повторная проверка выполняется отдельной командой `python revisit.py crawler.example.yaml`, только когда наступил назначенный срок. Эта команда не импортирует новые URL; после сбоя или остановки повторное задание остаётся за ней.

3 Результаты

3.1 Запуск и автотесты

Код робота запускается командой `python crawler.py crawler.example.yaml`. В примере ограничение `max_documents: 10` позволяет проверить небольшую часть списка `urls.txt`. Команда `python run_tests.py` проверяет систему без сервера MongoDB и без обращений к сайтам.

Успешно прошли 64 теста, в том числе 25 сценариев работы робота. Они охватывают сохранение HTML и времени Unix, продолжение после остановки, ответы `304` и `200`, контрольную сумму, `robots.txt` и `Retry-After`, а также отделение повторных проверок от первичной загрузки. Локальные HTML-образцы первой работы проверяются прежними тестами.

3.2 Пробная обкачка

MongoDB запускалась локально через Docker Compose. Сначала робот сохранил 24 проверочные страницы из `urls.txt`, затем выборка была расширена. На 8 октября 2026 года в очереди находилось 824 URL: 368 завершённых, 441 ожидаю-

щий, 14 исключённых и один оставшийся в обработке после остановки. Большой инвентарь в миллион URL в базу пока не импортировался.

Источник	Скачано	В корпусе	С текстом	Только заголовок
PlayGround	46	46	42	4
StopGame	51	51	51	0
IXBT.games	46	46	46	0
Игромания	36	36	36	0
GameMAG	54	54	53	1
PC Gamer	56	54	54	0
Eurogamer	39	36	36	0
GamingOnLinux	46	45	45	0
Всего	**374**	**368**	**363**	**5**

Все 374 записи `documents` имеют соответствующие страницы `raw_pages`; их контрольные суммы совпадают, пустого HTML и потерянных исходных страниц нет. Таблица отдельно показывает пригодные для индексации документы и страницы без основного текста, у которых можно использовать заголовок.

Первоначальные 124 записи были перенесены из формата со встроенным HTML в раздельные коллекции после резервного копирования базы. Средний размер записи в `documents` уменьшился примерно с 470 до 7 КБ, что удобнее для просмотра результатов разбора.

3.3 Репозиторий

Исходный код, конфигурации, тесты и отчёт размещены в ветке `lab2` репозитория: https://github.com/kirbasss/MAI-IR/tree/lab2. Сырые страницы и содержимое базы данных в GitHub не загружаются.



Рис. 1 — QR-код для перехода к ветке `lab2` репозитория.

4 Вывод

В этой работе я дополнил сбор игровых публикаций устойчивой очередью в MongoDB. Теперь робот может продолжить обход после остановки и периодически проверять уже сохранённые страницы. Было полезно разобраться, как связать состояние очереди с сохранением документа и зачем при повторной проверке нужны `ETag`, `Last-Modified` и контрольная сумма ответа.

Я проверил робота тестами и пробной загрузкой в MongoDB. На практике выяснилось, что даже успешный ответ сайта не всегда означает пригодный текстовый документ: в sitemap встречаются страницы категорий, а у некоторых публикаций иная разметка. Перед массовой обкачкой нужно следить за долей непригодных страниц и расходом места.

Приложение А. Примеры запросов к MongoDB

Ниже приведены читающие запросы к базе `mai_ir` и ответы на них после пробной обкачки 8 октября 2026 года. При продолжении загрузки числа изменятся.

А.1 Состояние очереди

```
db.frontier.aggregate([
  { $group: { _id: "$state", count: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
```

Ответ: `done: 368`, `excluded: 14`, `in_progress: 1`, `pending: 441`.

А.2 Интервал до повторной проверки

```
db.frontier.aggregate([
  { $match: { state: "done" } },
  { $group: {
      _id: { $subtract: ["$available_at", "$last_checked_at"] },
      count: { $sum: 1 }
  } }
])
```

Ответ: `[{_id: 2592000, count: 368}]`. Таким образом, для всех завершённых заданий следующая проверка назначена через 2 592 000 секунд, то есть через 30 дней после предыдущей.

А.3 Связь разобранных документов с HTML

```
db.documents.aggregate([
  { $lookup: {
      from: "raw_pages", localField: "raw_page_id",
      foreignField: "_id", as: "raw"
  } },
  { $unwind: "$raw" },
  { $match: { $expr: {
      $eq: ["$content_sha256",
            "$raw.content_sha256"]
  } } },
  { $count: "matched" }
])
```

Ответ: `[{matched: 374}]`. Каждый из 374 разобранных документов нашёл соответствующий сырой HTML с тем же хешем.

А.4 Документы без выделенного текста

```
db.documents.countDocuments(
  { "parsed.parse_error": { $type: "string" } })
```

Ответ: `6`. Все шесть записей исключены из будущего поискового индекса; у пригодных документов ошибок разбора нет.